

Lesson 4 Understanding and Creating Docker Image in Depth

1. Image Understanding

1. An image is a lightweight, executable, and self-sufficient software package, including all the necessary components for running software. The application and configurations are packaged into a complete, deliverable, and deployable running environment that contains code, runtime, required libraries, environment variables, configuration files, etc, which is the image file.

2. The docker containers can only be instantiated from image files.

2. UnionFS (Union File System)

1. UnionFS is a layered, lightweight, and high-performance file system that serves as the basis of the docker image. It allows for modifications to the file system to be added layer by layer and also enables different directories to be mounted onto the same virtual file system.

2. Images can be built through layering based on a base image to create various specific application images.

The characteristic of UnionFS is that it can simultaneously load multiple file systems, while appearing to the outside as a single file system. Union loading will overlay each file system to ensure that the final file system includes all layered files and directories.

3. Image Layering

When an image is downloaded, please observe the downloaded log output. You can see it is downloaded layer by layer.

```
pi@raspberrypi:~ $ docker pull mysql
Using default tag: latest
latest: Pulling from library/mysql
6988ac25ab22: Pull complete
cd2cec5cf6db: Pull complete
e703d567fd0e: Pull complete
de5675f5f5a6: Pull complete
28476d81f322: Pull complete
ccea770348da: Pull complete
842d004c6831: Pull complete
37f7eaf29320: Pull complete
03498da0acb4: Pull complete
00456c5c10e4: Pull complete
Digest: sha256:d7c20c5ba268c558f4fac62977f8c7125bde0630ff8946b08dde44135ef40df3
Status: Downloaded newer image for mysql:latest
docker.io/library/mysql:latest
```

Enter the command “docker image inspect mysql” in the terminal to check the image layering.

```
pi@raspberrypi:~ $ docker image inspect mysql
[
  {
    "Id": "sha256:b068ea59c6773ae58f27b4f289fecbb12cc416ba2d839736f8c62ad93d43fd49",
    "RepoTags": [
      "mysql:latest"
    ],
    "RepoDigests": [
      "mysql@sha256:d7c20c5ba268c558f4fac62977f8c7125bde0630ff8946b08dde44135ef40df3"
    ],
    "Layers": [
      "sha256:486ffb2b78bfe1b7de0674b28ecea6dfe79457f6fa4d03bd36d0879aeee68aa9",
      "sha256:df476ce8a77c66711cd397c3940177979afb8db8eb46153b0f7abd0525de9c519",
      "sha256:8fd3e8a3a23536b6cac42d2fa4ae230c99f84c2872bb4142d63db87d0a835f8d",
      "sha256:8d2f103f0e6304a4fa1b5898625e03ace744676a68666e1038b59561083c2077",
      "sha256:4eb705e240740b35d1d257ad084e1019fe6cc3f2729a31edf6e73ba50c7b2aab",
      "sha256:e2fbbef5f179aae0314176e2fd0a3428f3655d8cf3a5c2402cd659fead82ea38",
      "sha256:7b15982fa391b892c657e20948ac01855ad202353b5269ddc367ba386d220d9f",
      "sha256:4e6b6bd5aa87ba4b2774f3f0a06bb2f7600ffba2b9d235fc1583a53edbb970a2",
      "sha256:bf624f50d72ed948606a5b2ad612d9f62ae0f0e0e889a34f84aaef1f73cbf815",
      "sha256:3e43f9bac43a4acbf89b5fe45b493b9693670bd0fa67b7d0f6f13e7838a44c1e"
    ]
  }
]
```

3.1 Layering Understanding

All docker images start at a base image layer. When a modification or new component is added, a new image layer is created on top of the current image layer.

Illustrate by a simple example. If a new image is created based on “debian”, it will be the first layer of the new image. If a Python package is added to the image, the second image layer will be created on the top of the base image layer. If a security patch is added, the third image layer will be

created on the top of the second layer. The image will be built like this layer by layer.

All docker images are read-only. When a container is started, a new writable layer is mounted on the top of the image layers, which is commonly referred to as the container layer. The layers below it are called image layers.

3.2 Advantage of Layering in Docker Image

The advantage of layering is resource sharing. For example, if there are multiple images created from the same base image, the host only needs to keep one copy of the base image on its disk and in its memory. This allows the base image to be shared among all containers, and each layer of the image can be shared.

4. Image Creating

When the downloaded images from the docker image repository cannot meet your requirements, there are two ways to modify them.

① Use “docker commit” to update an image from the created container and commit the image.

② Use the “Dockerfile” command to create a new image.

4.1 Commit an Image from Container

Command explanation: `docker commit -m= “commit message” -a= “author” container ID target_image_name:[Tag]` 【“-m” and “-a” parameters can be omitted】

1) Press “Ctrl+Alt+T” to open the command line terminal and enter “docker ps -a”. It displays all previously run containers, and you need to locate the container ID to be committed.

```
pi@raspberrypi:~$ docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
04b86cdec880	hello-world	"/hello"	28 minutes ago	Exited (0) 28 minutes ago		hardcore_grothendieck
f4b0d06a2fe7	hello-world	"/bin/sh"	29 minutes ago	Created		charming_blackburn
067b778e4486	hello-world	"/bin/bash"	30 minutes ago	Created		reverent_wiles
f16dc4876948	debian	"/bin/bash"	31 minutes ago	Exited (0) 30 minutes ago		practical_hodgkin
6c8e3babad11	hello-world	"/bin/bash"	31 minutes ago	Created		happy_jennings
a450933f3aa0	hello-world	"/bin/bash"	32 minutes ago	Created		flamboyant_mayer
eab350ee35da	hello-world	"/bin/bash"	34 minutes ago	Created		nifty_edison
d2408c2fb32f	hello-world	"/hello"	41 minutes ago	Exited (0) 41 minutes ago		dreamy_spence
efe969b704a7	debian	"/bin/bash"	3 hours ago	Exited (137) 33 minutes ago		fervent_williams
004c7b83e76c	debian	"/bin/bash"	3 hours ago	Exited (0) 3 hours ago		quirky_mccarthy
cd1df225f0bf	debian	"/bin/bash"	3 hours ago	Exited (137) 33 minutes ago		nostalgic_black
109081ef693e	debian	"/bin/bash"	3 hours ago	Exited (255) 3 hours ago		xenodochial_cartwright
dccc090bee5b	debian	"/bin/bash"	3 hours ago	Exited (0) 3 hours ago		gifted_poincare
56d746b94607	debian	"/bin/bash"	3 hours ago	Exited (255) 3 hours ago		optimistic_kirch
2672149d25ba	debian	"bash"	3 hours ago	Exited (0) 3 hours ago		great_wilson
9aeb4cc795e6	debian	"/bin/bash"	4 hours ago	Exited (0) 4 hours ago		intelligent_ramanujan

2) Enter “docker commit 9aeb4cc795e6 debain:1.0” in the terminal to commit the image of container “9aeb4cc795e6” and name it “debain”. The tag name is 1.0.

```
pi@raspberrypi:~$ docker commit 9aeb4cc795e6 debain:1.0
sha256:965cecd53c559bcce980346a3439f01555b61396ccc3b023323d77a6cb64096d
```

3) Enter “docker images” in the terminal to view the images on the master. You can see a “debain” has been added to the list of images.

```
pi@raspberrypi:~$ docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
debain	1.0	965cecd53c55	5 minutes ago	139MB
test/debian	v1	7f96d767c7be	16 hours ago	139MB
mysql	latest	b068ea59c677	4 days ago	638MB
debian	latest	60e0ade36b1a	12 days ago	139MB
hello-world	latest	ee301c921b8a	8 months ago	9.14kB

4.2 “dockerfile” Image Creating

Command explanation: docker build “-f dockerfile” means file path and “-t” is the name of the new image: TAG . # docker build. The final “.” in the command indicates the current directory.

Use the command “docker build” to create a new image from scratch. As a result, a “Dockerfile ” file needs to be created, including a set of commands for Docker on how to build the image.

About the compiling of “dockerfile”, please refer to “https://docs.docker.com/develop/develop-images/dockerfile_best-practices/”.

1) Press “Ctrl+Alt+T” to open the command line terminal. Enter “vim Dockerfile” in the terminal to create and open the file.

```
pi@raspberrypi:~ $ vim Dockerfile
```

2) Type the content shown below in the terminal and press “ESC”. Enter “:wq” to exit and save the file.

```
File Edit Tabs Help
FROM      debian
MAINTAINER Fisher "fisher@sudops.com"
RUN        /bin/echo 'root:123456' |chpasswd
RUN        useradd runoob
RUN        /bin/echo 'runoob:123456' |chpasswd
RUN        /bin/echo -e "LANG=\"en_US.UTF-8\"" >/etc/default/locale
EXPOSE     22
EXPOSE     80
CMD        /usr/sbin/sshd -D
```

```
:wq
```

3) Enter “docker build -f dockerfile -t debain:1.1 .” in the terminal to build a new image.

```
pi@raspberrypi:~ $ docker build -f Dockerfile -t debian:1.1 .
[+] Building 14.3s (9/9) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile              0.2s
=> => transferring dockerfile: 318B                             0.0s
=> [internal] load metadata for docker.io/library/debian:latest 0.0s
=> [internal] load .dockerignore                                0.3s
=> => transferring context: 2B                                    0.0s
=> [1/5] FROM docker.io/library/debian:latest                  0.5s
=> [2/5] RUN /bin/echo 'root:123456' |chpasswd                  2.5s
=> [3/5] RUN useradd runoob                                     2.8s
=> [4/5] RUN /bin/echo 'runoob:123456' |chpasswd                2.5s
=> [5/5] RUN /bin/echo -e "LANG=\"en_US.UTF-8\"" >/etc/default/locale 2.9s
=> exporting to image                                           1.2s
=> => exporting layers                                          1.2s
=> => writing image sha256:30555f5a4f74883ee7441379af1bc9230163721edf4e4 0.0s
=> => naming to docker.io/library/debian:1.1                   0.0s
```

4) Enter “docker images” in the terminal, then you can see the newly built

image.

```
pi@raspberrypi:~ $ docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
debian	1.1	30555f5a4f74	19 minutes ago	139MB
test/debian	v1	7f96d767c7be	17 hours ago	139MB
mysql	latest	b068ea59c677	4 days ago	638MB
debian	latest	60e0ade36b1a	12 days ago	139MB
ros	melodic	3365d517511a	6 weeks ago	1.2GB
hello-world	latest	ee301c921b8a	8 months ago	9.14kB